

Bridging the Digital Divide: Understanding Application Programming Interfaces

APIs are fundamental building blocks of the digital world, acting as crucial bridges that enable communication and data exchange between diverse software systems. Let's delve into their significance in modern software development, including their role in microservices and event-driven architecture (EDA).



Digitisation of society has brought immense benefits but has also highlighted and, in some cases, exacerbated the disparities in access to technology and its advantages. At the heart of modern digital infrastructure lie application programming interfaces (APIs), often unseen but critical conduits of information and functionality. Understanding their role is paramount to fostering greater digital equality. For developers and tech enthusiasts, leveraging APIs can significantly enhance the functionality and efficiency of their applications.

API is a software intermediary that enables two distinct applications to communicate and exchange

data with each other. In essence, APIs function as precisely defined contracts or bridges that specify the rules and protocols for how software components should interact. This allows developers to access and utilise the functionalities or data of other software platforms and services natively within the applications they are building, without needing to understand the intricate internal workings of those external systems. APIs operate through a request-response mechanism between a client and a server. The client or the application making the request calls the server through the API. The server then processes this request and sends back the appropriate response. For

example, when you use a third-party payment system like PayPal on an e-commerce site, an API call is made to process the payment. The API facilitates secure and efficient communication between the e-commerce site and PayPal, ensuring that transactions are completed seamlessly.

The core components of an API define this interaction. These include:

API specification: A structured description detailing the API's capabilities and how to interact with it, outlining available functions, parameters, and expected responses.

Endpoints: Specific uniform resource locators (URLs) that designate where requests should be sent to access particular API functionalities.